

GLA UNIVERSITY

Department of Computer Engineering & Applications

PROJECT SYNOPSIS CUM REPORT

PROJECT TITLE

AgentForge: Enterprise Multi-Agent SDLC Factory

Name of Student	Maulik Verma
University Roll No.	22515500061
Branch	CSE AIML
Semester	3rd sem
Course	B.Tech
Submitted To	Mr. Vansh Aggarwal, ML & Data Visualisation Faculty
Submitted By	Maulik Verma
Academic Year	2025-2029
Date of Submission	16/08/2026

A Synopsis cum Report Submitted in Partial Fulfilment of the ML Software Project

Instructions: Replace every [bracketed] placeholder above with your actual details before submission.

INTRODUCTION

Introduction to the Project

AgentForge is a proposed enterprise-oriented software development platform that uses multiple AI agents to coordinate different stages of the Software Development Life Cycle (SDLC). The main idea is to move from a single AI assistant that writes code on request to a structured group of specialized agents that work together under a central supervisor.

The project belongs to the area of AI-assisted software engineering and developer productivity. In a normal development workflow, requirements are discussed first, technical specifications are prepared, work is divided into tickets, developers implement the tasks, and the resulting software is tested and deployed. These steps often require repeated manual coordination between planning, development, testing, and DevOps activities. AgentForge proposes to connect these stages through a common orchestration layer.

The planned system starts with a human user providing requirements in natural language. A Supervisor Agent interprets the request and coordinates a System Architect and Product Manager. The architecture and business requirements are then converted into technical blueprints, epics, and actionable tickets. Domain Team Leads divide the work among specialized worker agents for areas such as frontend, backend, and AI/ML. The workers are intended to generate and commit code inside isolated Docker environments.

Testing is planned as a separate stage using specialized QA agents. UI, API, and load testing can trigger a feedback loop when problems are found, sending bug reports back to the planning stage instead of allowing a failed build to continue toward deployment. After successful testing, a DevOps Principal Agent prepares the application for staging. The final staging step intentionally includes Human-in-the-Loop approval before production deployment.

The overall goal of AgentForge is to create a controlled and traceable multi-agent workflow that can automate repetitive SDLC coordination while keeping important technical and deployment decisions reviewable by a human.

PROBLEM STATEMENT**Statement of the Problem**

AgentForge addresses a coordination problem in modern software development: requirements, architecture, planning, coding, testing, and deployment are often handled through separate tools and roles. As work moves between these stages, information can be interpreted differently, dependencies can remain unresolved, and integration issues may be discovered late. The burden increases when frontend, backend, AI/ML, QA, and DevOps activities have to progress together.

A conventional AI coding assistant can help with individual tasks, but it does not by itself provide a structured workflow for turning requirements into technical plans, distributing work, running validation, and routing failures back to the right stage. AgentForge proposes a multi-agent approach in which a Supervisor coordinates the System Architect and Product Manager, Domain Team Leads delegate implementation work to specialized agents, and an MCP integration layer connects project/ticket management, source control, file access, and infrastructure services.

The project also considers the risks of automation. Planned isolation through Docker, automated QA feedback, controlled handoffs, and Human-in-the-Loop staging approval are intended to keep the workflow traceable and reviewable. The aim is to automate repetitive coordination without removing human responsibility for important technical and deployment decisions.

PROBLEM STATEMENT**Scope & Areas of Impact**

- Software Engineering — Reduces repetitive coordination between requirements, planning, implementation, testing, and deployment.
- Enterprise Product Development — Helps organize feature requests into structured technical work and development tasks.
- AI-Assisted Development — Uses specialized agents for different software engineering activities instead of relying on one general-purpose agent.
- Quality Assurance — Automates planned UI, API, and load-test execution and routes failures back into development.
- DevOps & Deployment — Coordinates containerization and staging deployment while retaining a human approval point before production.

PROBLEM STATEMENT**Target Stakeholders & Segment Analysis**

Primary stakeholders

- Software teams — Need a more organized way to coordinate AI-assisted development work.
- Product managers — Benefit from turning requirements and technical plans into actionable epics and tickets.
- Developers — Can receive smaller, domain-specific tasks from team leads and worker agents.
- QA engineers — Can use automated test agents to identify and report failures earlier.
- DevOps engineers — Can receive tested artifacts for containerization and staging.

Initial user segments

- Small software teams: limited coordination capacity; may use AgentForge to automate routine SDLC handoffs.
- Student / prototype development teams: need a structured workflow for experimenting with multi-agent development.
- Enterprise engineering teams: may use the architecture as a foundation for controlled AI-assisted development.

The project is initially intended as a prototype rather than a production product, so exact market size and commercial adoption are not being claimed at the synopsis stage.

PROPOSED SOLUTION**Minimum Viable Product (MVP) Definition**

MVP scope — only the core workflow needed to demonstrate the idea:

- Requirement intake — Accept a software requirement from a human user.
- Central orchestration — Use a Supervisor/PM flow to coordinate the planned SDLC stages.
- Technical planning — Generate a basic technical blueprint, API considerations, and database plan.
- Ticket planning — Convert the plan into structured development tasks through the planned MCP integration.
- Domain delegation — Route tasks to Frontend, Backend, and AI/ML team leads and their worker agents.
- Isolated code generation — Allow worker agents to generate code in Docker-based environments.
- Basic QA loop — Run planned UI/API tests and return failures as bug reports.
- Staging handoff — Containerize the tested artifact and prepare a staging deployment with Human-in-the-Loop approval.

Not in the MVP: full production-scale infrastructure, broad enterprise authentication, advanced analytics, large-scale model training, and unrestricted autonomous production deployment.

PROPOSED SOLUTION**Product Overview - Concept & Value Proposition**

AgentForge is a proposed web-based control platform for an AI-assisted software development workflow. A user provides a software requirement, and the platform coordinates a set of specialized agents that handle planning, development, testing, and deployment preparation.

Instead of treating software generation as one large prompt, the system divides the work into roles. A Supervisor coordinates the process, an Architect prepares the technical direction, a Product Manager converts the plan into development work, and domain-specific agents handle implementation. QA and DevOps agents then operate on the resulting build.

The main value proposition is structured automation: each agent has a defined responsibility, the workflow has clear handoffs, and the process can stop for human review before production.

PROPOSED SOLUTION**Product Overview - Working Principle & Differentiators**

The planned workflow follows a controlled sequence. First, the user submits requirements to the Supervisor. The Supervisor coordinates the Architect and PM to create technical blueprints and actionable epics. These are passed through the planned MCP integration layer to the ticket and file systems.

Domain Team Leads then break the work into smaller tasks for specialized worker agents. Workers are expected to generate code inside isolated Docker environments and commit approved changes. A successful integrated commit triggers the QA stage.

Cypress, Postman, and K6 are planned as specialized QA agents for UI, API, and load testing. If a test fails, a bug report is routed back to the PM and the development loop starts again. If testing passes, DevOps prepares a containerized staging deployment. The system then pauses for Human-in-the-Loop review.

Key differentiators:

- Role-based multi-agent workflow rather than one general coding agent.
- Explicit feedback loop from QA to planning and development.
- Isolated execution environments for generated code.
- MCP-based connections to development tools and project data.
- Human approval before production deployment.

PROPOSED SOLUTION**Rationale & Market Need**

The need for AgentForge comes from the coordination overhead in software development. AI coding tools can generate useful code, but a complete software project also requires requirements analysis, architecture, task planning, testing, version control, and deployment. These stages are not automatically connected simply because code can be generated.

Existing AI coding assistants generally focus on individual developer tasks such as generating, explaining, or modifying code. Traditional project-management and CI/CD tools provide strong workflow support, but they do not by themselves provide a coordinated set of AI agents for planning and implementation.

AgentForge is proposed to connect these activities into one controlled workflow. Its intended benefit is not to remove developers from the process, but to reduce repetitive handoffs and make AI-generated work easier to track, test, and review. The Human-in-the-Loop checkpoint is therefore a deliberate part of the design rather than an optional extra.

PROPOSED SOLUTION**Target Audience & User Personas**

Primary audience

- Software developers and small engineering teams working with AI-assisted development.
- Product managers or technical leads who need structured conversion of requirements into development tasks.
- QA/DevOps-oriented users interested in automating repetitive validation and deployment preparation.

Persona 1 — Aarav, 24, Software Developer

Need: Quickly turn planned features into implementation tasks and usable code.

Pain point: Switching repeatedly between requirements, tickets, coding, testing, and debugging.

Expected value: Specialized agents handle routine parts of the workflow while Aarav reviews the important outputs.

Persona 2 — Neha, 29, Product/Technical Lead

Need: Keep a software request connected to its technical plan and development progress.

Pain point: Requirements can become fragmented across documents, tickets, repositories, and developer discussions.

Expected value: A central orchestration flow provides clearer handoffs and a traceable path from requirement to tested build.

Primary segment: software development teams. Secondary segment: educational/prototype teams exploring agentic software engineering.

PROPOSED SOLUTION**Product Walkthrough - Structure & Scope**

Planned product walkthrough structure

1. Requirement Intake — User enters the initial software requirement.
2. Project/Plan View — Supervisor and planning stages show the current project state.
3. Technical Blueprint — Architecture and technical specifications generated from the requirement.
4. Epic & Ticket Plan — PM converts the blueprint into actionable development work.
5. Agent Task Board — Tasks are delegated to domain leads and worker agents.
6. Code Generation Workspace — Generated changes and commits are reviewed.
7. QA Status — UI, API, and load-test results are shown.
8. Bug Feedback Loop — Failed tests are converted into development feedback.
9. Deployment View — Tested artifacts move toward staging.
10. HITL Review — Human approval is required before production.

These are planned screens for the future implementation; no completed product screenshots are being claimed in this synopsis.

PROPOSED SOLUTION - PRODUCT WALKTHROUGH

Product Walkthrough - Screen 1

Planned Screen 1 — Requirement Intake / Project Creation

This screen will be the starting point of the AgentForge workflow. The user will provide a project description, major requirements, and relevant preferences. The Supervisor Agent will receive this information and begin the planning flow.

Because the project is currently at the synopsis stage, this page describes the intended screen rather than presenting a fabricated implementation screenshot.

PLANNED SCREEN / WIREFRAME AREA

No implementation screenshot is claimed at synopsis stage. This area is reserved for the Requirement Intake screen to be developed during the project.

Feature / Screen Name: _____

Explanation (2-3 lines): _____

PROPOSED SOLUTION - PRODUCT WALKTHROUGH**Product Walkthrough - Screen 2****WHAT TO DO ON THIS PAGE**

Insert a clear, full-size screenshot of your product (Screenshot 2) below.

Add a short caption naming the screen/feature shown.

Add 2-3 lines explaining what the user is doing/seeing on this screen and why it matters.

[Insert Screenshot 2 Here]

Feature / Screen Name: _____

Explanation (2-3 lines): _____

PROPOSED SOLUTION - PRODUCT WALKTHROUGH

Product Walkthrough - Screen 3

WHAT TO DO ON THIS PAGE

Insert a clear, full-size screenshot of your product (Screenshot 3) below.

Add a short caption naming the screen/feature shown.

Add 2-3 lines explaining what the user is doing/seeing on this screen and why it matters.

[Insert Screenshot 3 Here]

Feature / Screen Name: _____

Explanation (2-3 lines): _____

PROPOSED SOLUTION - PRODUCT WALKTHROUGH

Product Walkthrough - Screen 4

WHAT TO DO ON THIS PAGE

Insert a clear, full-size screenshot of your product (Screenshot 4) below.

Add a short caption naming the screen/feature shown.

Add 2-3 lines explaining what the user is doing/seeing on this screen and why it matters.

[Insert Screenshot 4 Here]

Feature / Screen Name: _____

Explanation (2-3 lines): _____

PROPOSED SOLUTION - PRODUCT WALKTHROUGH**Product Walkthrough - Screen 5****WHAT TO DO ON THIS PAGE**

Insert a clear, full-size screenshot of your product (Screenshot 5) below.

Add a short caption naming the screen/feature shown.

Add 2-3 lines explaining what the user is doing/seeing on this screen and why it matters.

[Insert Screenshot 5 Here]

Feature / Screen Name: _____

Explanation (2-3 lines): _____

PROPOSED SOLUTION - PRODUCT WALKTHROUGH

Product Walkthrough - Screen 6

WHAT TO DO ON THIS PAGE

Insert a clear, full-size screenshot of your product (Screenshot 6) below.

Add a short caption naming the screen/feature shown.

Add 2-3 lines explaining what the user is doing/seeing on this screen and why it matters.

[Insert Screenshot 6 Here]

Feature / Screen Name: _____

Explanation (2-3 lines): _____

PROPOSED SOLUTION - PRODUCT WALKTHROUGH**Product Walkthrough - Screen 7****WHAT TO DO ON THIS PAGE**

Insert a clear, full-size screenshot of your product (Screenshot 7) below.

Add a short caption naming the screen/feature shown.

Add 2-3 lines explaining what the user is doing/seeing on this screen and why it matters.

[Insert Screenshot 7 Here]

Feature / Screen Name: _____

Explanation (2-3 lines): _____

PROPOSED SOLUTION - PRODUCT WALKTHROUGH

Product Walkthrough - Screen 8

WHAT TO DO ON THIS PAGE

Insert a clear, full-size screenshot of your product (Screenshot 8) below.

Add a short caption naming the screen/feature shown.

Add 2-3 lines explaining what the user is doing/seeing on this screen and why it matters.

[Insert Screenshot 8 Here]

Feature / Screen Name: _____

Explanation (2-3 lines): _____

PROPOSED SOLUTION - PRODUCT WALKTHROUGH

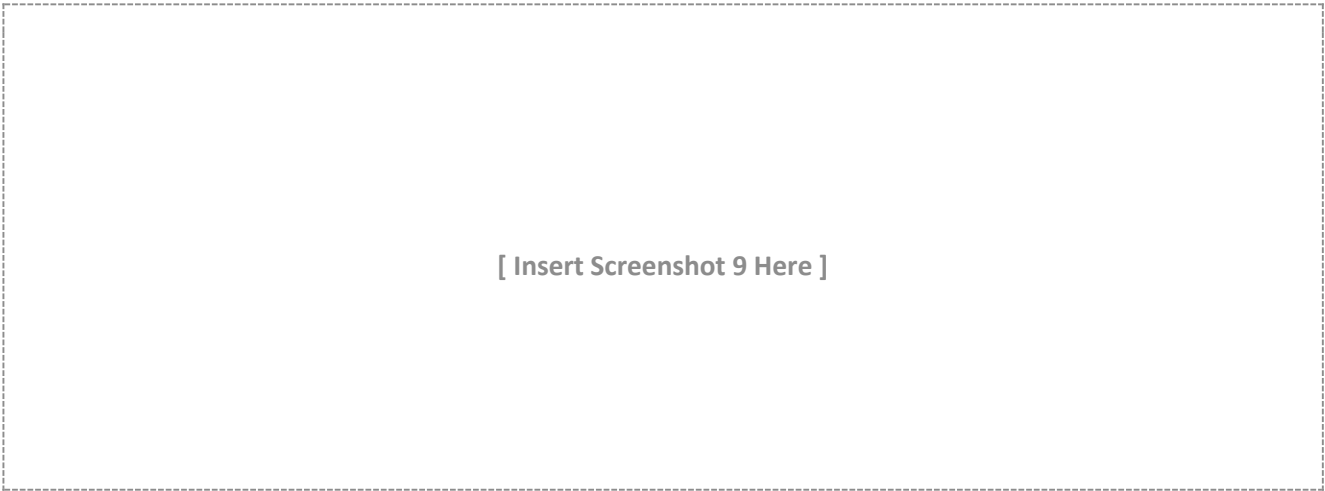
Product Walkthrough - Screen 9

WHAT TO DO ON THIS PAGE

Insert a clear, full-size screenshot of your product (Screenshot 9) below.

Add a short caption naming the screen/feature shown.

Add 2-3 lines explaining what the user is doing/seeing on this screen and why it matters.



Feature / Screen Name: _____

Explanation (2-3 lines): _____

PROPOSED SOLUTION - PRODUCT WALKTHROUGH

Product Walkthrough - Screen 10

WHAT TO DO ON THIS PAGE

Insert a clear, full-size screenshot of your product (Screenshot 10) below.

Add a short caption naming the screen/feature shown.

Add 2-3 lines explaining what the user is doing/seeing on this screen and why it matters.

[Insert Screenshot 10 Here]

Feature / Screen Name: _____

Explanation (2-3 lines): _____

PROPOSED SOLUTION - PRODUCT WALKTHROUGH

Product Walkthrough - Screen 11

WHAT TO DO ON THIS PAGE

Insert a clear, full-size screenshot of your product (Screenshot 11) below.

Add a short caption naming the screen/feature shown.

Add 2-3 lines explaining what the user is doing/seeing on this screen and why it matters.

[Insert Screenshot 11 Here]

Feature / Screen Name: _____

Explanation (2-3 lines): _____

PROPOSED SOLUTION

Key Functionalities - Core Features

WHAT TO DO ON THIS PAGE

List the core functionalities of your product as bullet points.

For each functionality, add a one-line description of what it does.

Group related functionalities under sub-headings if helpful.

PROPOSED SOLUTION**Key Functionalities - Extended Features**

Extended / Secondary Features

- Agent activity log — Records important agent actions and handoffs for traceability.
- Workflow status — Shows which SDLC stage is currently active and whether a task is waiting, running, failed, or approved.
- Repository and ticket synchronization — Keeps generated work connected to the relevant code and project-management records.
- QA result history — Stores test outcomes and links failures to the corresponding development task.
- Human approval controls — Provides an explicit approval/rejection step at staging.
- Basic project analytics — Planned summaries such as completed tasks, failed tests, and workflow status.
- Notification hooks — Planned alerts for failed QA runs, blocked workflows, and approval requests.
- Agent isolation controls — Keeps generated code execution separated through Docker-based environments.

PROPOSED SOLUTION

Key Functionalities - Future Roadmap

WHAT TO DO ON THIS PAGE

List functionalities planned for future versions, beyond this project's timeline.

Briefly explain why these were deferred (time, complexity, data availability, etc).

This shows evaluators you have a long-term vision for the product.

TECHNICAL ARCHITECTURE & DESIGN

Technology Stack Overview

WHAT TO DO ON THIS PAGE

List all technologies used: frontend, backend, database, AI/ML libraries, cloud/hosting, tools.

Present as a clean table: Layer | Technology | Purpose/Reason for choosing it.

Keep this to ONE page - be concise, details go in later sections.

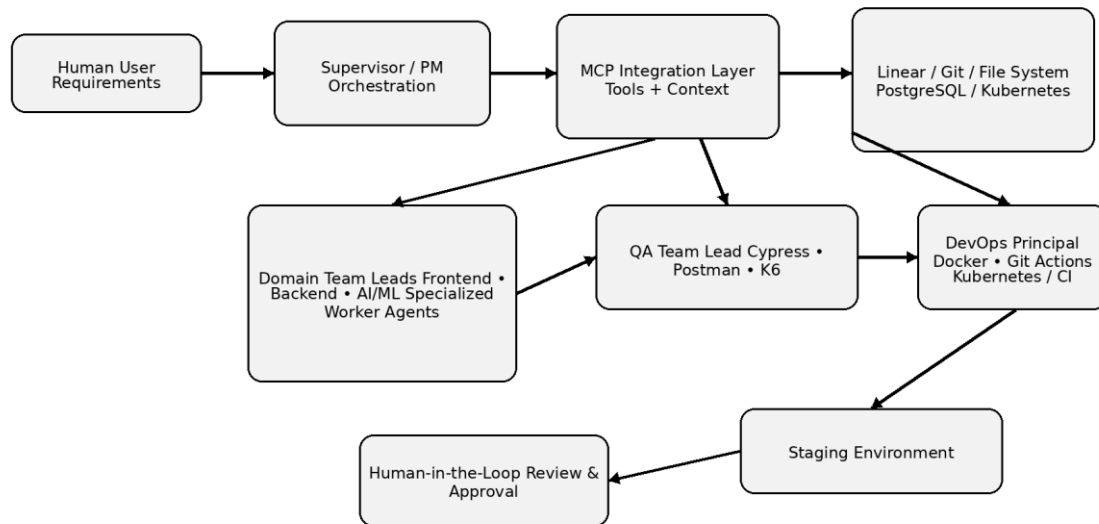
Layer	Technology Used	Purpose / Reason

TECHNICAL ARCHITECTURE & DESIGN

System Architecture Diagram (Part 1 of 2)

Architecture overview: the system is organized around a central LangGraph orchestration layer, with MCP providing controlled access to tickets, repositories, files and infrastructure. Domain agents handle implementation, while QA and DevOps form the validation and deployment stages.

AgentForge — High-Level System Architecture



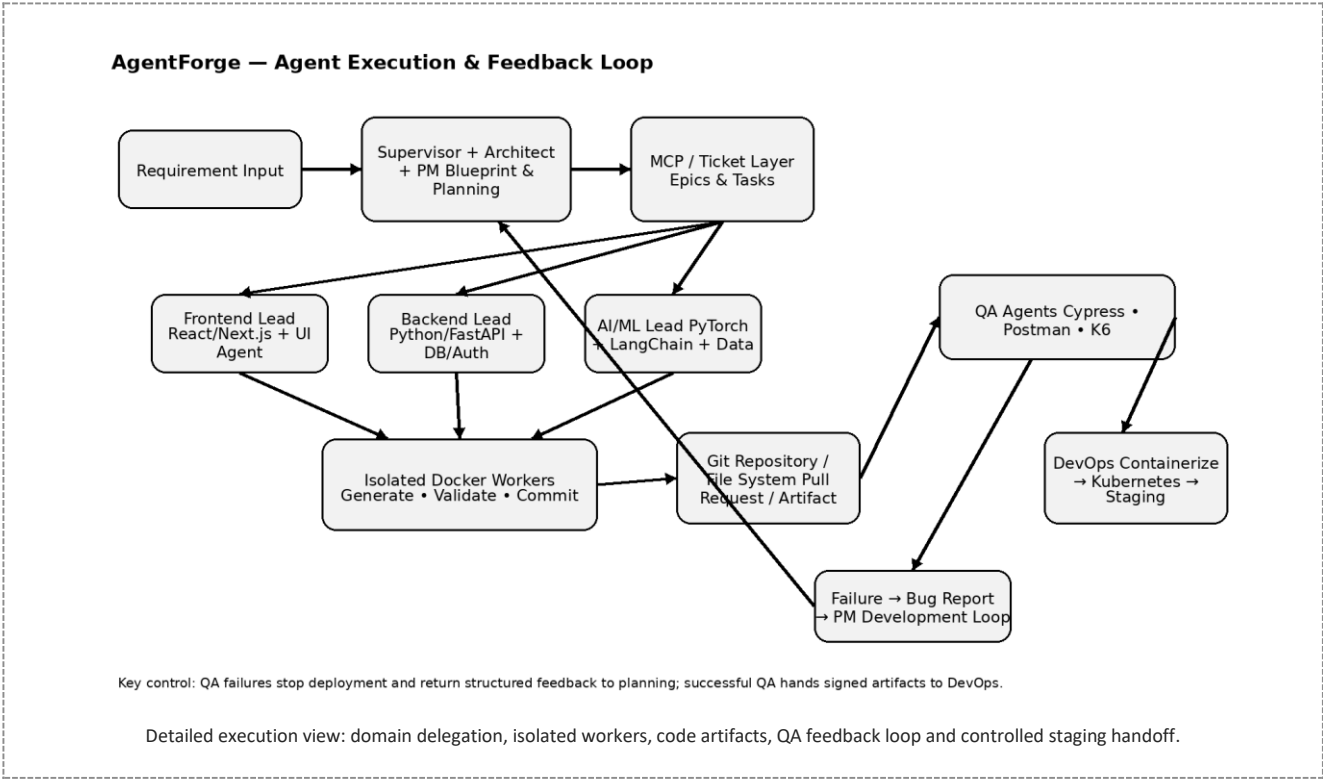
Flow: Requirements → Orchestration → Tool Context → Domain Work → QA → DevOps → Staging → Human Approval

High-level architecture: orchestration, MCP integrations, domain agents, QA, DevOps, staging and human approval.

TECHNICAL ARCHITECTURE & DESIGN

System Architecture Diagram (Part 2 of 2)

Execution detail: domain leads delegate work to specialized agents running in isolated Docker environments. Approved artifacts move through Git/file storage into QA. Failed tests return structured bug reports to the PM; successful tests are handed to DevOps for staging.



TECHNICAL ARCHITECTURE & DESIGN

Database Design & Schema Overview

WHAT TO DO ON THIS PAGE

Explain your database choice (SQL/NoSQL) and reasoning.

List major tables/collections with a short description of each and key fields.

Mention relationships between tables/collections (one-to-many, many-to-many, etc.).

TECHNICAL ARCHITECTURE & DESIGN**UML Diagram - System Design Representation****WHAT TO DO ON THIS PAGE**

Insert a relevant UML diagram (Use Case, Class, Sequence, or Activity diagram).

Choose the diagram type that best represents your system's logic/flow.

Add a 2-3 line note below explaining what the diagram represents.

[Insert UML Diagram Here]

ARTIFICIAL INTELLIGENCE & MACHINE LEARNING COMPONENT**AI/ML Model Overview**

AgentForge is not planned as a project that trains a new machine-learning model from scratch. Its AI/ML component is primarily an agentic software-engineering layer built around large language models (LLMs).

The planned system uses LLM-powered agents for tasks such as requirement interpretation, technical planning, ticket decomposition, code generation, debugging feedback, and coordination. LangGraph is proposed as the central orchestration framework because the workflow needs explicit state, sequential stages, branching, and feedback loops.

The learning paradigm is therefore not a conventional supervised/unsupervised training pipeline for a project-specific prediction model. The project will focus on using existing foundation models through structured agent roles, prompts, tools, and controlled execution environments. Model choice and exact provider will be finalized during implementation based on access, cost, latency, and coding performance.

ARTIFICIAL INTELLIGENCE & MACHINE LEARNING COMPONENT

Data Collection & Preprocessing

WHAT TO DO ON THIS PAGE

Describe the dataset(s) used - source, size, and format.

Explain preprocessing/cleaning steps applied (handling missing data, normalization, etc).

Mention any data augmentation or feature engineering performed.

ARTIFICIAL INTELLIGENCE & MACHINE LEARNING COMPONENT**Model Selection & Architecture**

Model / Agent Selection & Architecture

The proposed AI layer will use an LLM as the reasoning and code-generation engine, with LangGraph coordinating the individual agent roles. The architecture is intentionally role-based rather than using one agent for every task.

Planned agent structure:

- Supervisor Agent — receives the user's request and controls the overall workflow.
- System Architect Agent — converts requirements into technical specifications, API considerations, and database design.
- Product Manager Agent — turns the approved plan into epics and actionable tickets.
- Domain Team Leads — coordinate Frontend, Backend, and AI/ML implementation.
- Worker Agents — specialize in React/Next.js, UI/Tailwind, Python/FastAPI, Node.js/Auth, database work, PyTorch/LangChain, and data preparation.
- QA Agents — Cypress, Postman, and K6 based testing roles.
- DevOps Principal Agent — manages containerization and deployment preparation.

LangGraph is preferred for orchestration because the project requires stateful routing and loops, especially when QA failures must return to the PM/development stage. Exact LLM model and parameters will be selected during implementation rather than claimed in advance.

ARTIFICIAL INTELLIGENCE & MACHINE LEARNING COMPONENT**Model Training & Evaluation**

Model Training & Evaluation Plan

No model training or final evaluation results are being claimed at the synopsis stage because implementation has not started.

Planned evaluation will focus on the usefulness and reliability of the agent workflow rather than training accuracy. The prototype will be tested using representative software requirements and will be evaluated on:

- Requirement-to-blueprint consistency.
- Correctness of task decomposition and delegation.
- Code generation success rate for selected tasks.
- Percentage of generated changes that pass the planned QA checks.
- Bug-loop effectiveness — whether reported failures return to the correct development stage.
- Workflow completion time and number of human interventions.

During implementation, test cases will be documented and repeated so that improvements can be compared across iterations. The main expected challenges are inconsistent LLM outputs, incomplete requirements, tool-call failures, and generated code that requires human correction.

ARTIFICIAL INTELLIGENCE & MACHINE LEARNING COMPONENT**System Integration of the Model**

System Integration of the AI Layer

The AI layer will be integrated as part of the central AgentForge orchestration service. LangGraph will maintain the workflow state and route tasks between the Supervisor, Architect, PM, domain leads, QA, and DevOps agents.

Planned request flow:

1. User submits requirements to the application.
2. The Supervisor creates the initial workflow state.
3. Architect and PM agents enrich the request with technical and planning information.
4. MCP-connected tools provide access to tickets, repositories, files, and infrastructure context.
5. Worker agents generate code inside isolated Docker environments.
6. Approved commits trigger the QA stage.
7. QA results are returned to the orchestration layer; failures create a bug-report loop.
8. Successful QA produces a handoff to DevOps for staging.

FastAPI is planned for service/API integration where a backend interface is required. The exact service boundaries will be finalized during implementation. Human approval remains outside the autonomous loop at the staging-to-production boundary.

ARTIFICIAL INTELLIGENCE & MACHINE LEARNING COMPONENT**Performance Evaluation, Limitations & Future Enhancements**

Performance Evaluation, Limitations & Future Enhancements

Since AgentForge has not yet been implemented, final performance numbers are intentionally not provided. The first prototype will establish baseline measurements for task completion, code acceptance, QA pass rate, latency, and human intervention.

Expected limitations:

- LLM output can vary between runs and may require review.
- Ambiguous requirements can lead to incorrect plans or tickets.
- Tool integrations may fail or return incomplete context.
- Generated code may compile but still contain logical or security issues.
- Running several agents and test environments can require significant compute and time.
- Full production autonomy is intentionally outside the initial scope.

Planned future enhancements:

- Better project memory and retrieval of previous decisions.
- More specialized agents for security, documentation, and database migration.
- Stronger automated code review and policy checks.
- Improved observability and cost/latency monitoring.
- More robust recovery from failed tool calls.
- Expanded deployment targets after the staging workflow is validated.

ARTIFICIAL INTELLIGENCE & MACHINE LEARNING COMPONENT**Model Workflow / Pipeline Diagram****WHAT TO DO ON THIS PAGE**

Insert a workflow/pipeline diagram of your AI/ML model below.

Show stages: Data Input -> Preprocessing -> Model -> Output -> Integration with App.

Keep the diagram simple and readable, use consistent shapes for each stage.

[Insert Model Workflow Diagram Here]

PROJECT EXECUTION TIMELINE

Execution Timeline - Overview (August to November)

Execution Timeline — August to November

The project is planned as a phased prototype. August will focus on understanding the required tools and establishing the orchestration foundation. September will build the core multi-agent development workflow and MCP connections. October will concentrate on QA feedback loops and DevOps/staging integration. November will be used for integration testing, refinement, documentation, and the final Human-in-the-Loop demonstration.

Milestones

- Orchestration foundation — August — LangGraph workflow, Supervisor and PM structure.
- Tool/MCP integration — August–September — Connect planned ticket, Git/file, and project-context tools.
- Domain agent layer — September — Build Frontend, Backend, and AI/ML team leads and worker-agent flow.
- Isolated code generation — September–October — Docker execution, validation, and commit workflow.
- QA feedback loop — October — Cypress/Postman/K6 testing and automated bug routing.
- DevOps & staging — October–November — Containerization, Kubernetes/deployment pipeline, staging handoff.
- Final integration & HITL review — November — End-to-end testing, fixes, documentation, and human approval demonstration.

The timeline is a target plan, not a claim that these components have already been completed.

Milestone	Target Month	Status / Notes
Orchestration foundation	August	Planned — LangGraph workflow, Supervisor and PM structure
MCP/tool integration	Aug–Sep	Planned — ticket, Git/file and project-context connections
Domain agent layer	September	Planned — Frontend, Backend and AI/ML leads + worker flow
Isolated code generation	Sep–Oct	Planned — Docker execution, validation and commit workflow
QA feedback loop	October	Planned — Cypress, Postman and K6 testing with bug routing
DevOps & staging	Oct–Nov	Planned — containerization, Kubernetes/deployment pipeline
Final integration & HITL	November	Planned — end-to-end testing, refinement and final review

PROJECT EXECUTION TIMELINE

Month 1 - August

WHAT TO DO ON THIS PAGE

List all tasks planned for this month with estimated time required (in weeks).

Mark each task as 'Build' (development work) or 'Learn' (new skill/concept to study).

Be realistic - a month usually has about 4 weeks of effective working time.

Task	Type (Build/Learn)	Time Required (Weeks)

PROJECT EXECUTION TIMELINE

Month 2 - September

WHAT TO DO ON THIS PAGE

List all tasks planned for this month with estimated time required (in weeks).

Mark each task as 'Build' (development work) or 'Learn' (new skill/concept to study).

Be realistic - a month usually has about 4 weeks of effective working time.

Task	Type (Build/Learn)	Time Required (Weeks)

PROJECT EXECUTION TIMELINE

Month 3 - October

WHAT TO DO ON THIS PAGE

List all tasks planned for this month with estimated time required (in weeks).

Mark each task as 'Build' (development work) or 'Learn' (new skill/concept to study).

Be realistic - a month usually has about 4 weeks of effective working time.

Task	Type (Build/Learn)	Time Required (Weeks)

PROJECT EXECUTION TIMELINE

Month 4 - November

WHAT TO DO ON THIS PAGE

List all tasks planned for this month with estimated time required (in weeks).

Mark each task as 'Build' (development work) or 'Learn' (new skill/concept to study).

Be realistic - a month usually has about 4 weeks of effective working time.

Task	Type (Build/Learn)	Time Required (Weeks)

CITATIONS & DATA SOURCES**Citations - Datasets Used**

WHAT TO DO ON THIS PAGE

List every source under the category "Datasets Used" that you used for data in your project.

Format: Title of source, Author/Organization, Year, and full URL/link.

Only include sources you have actually referred to - do not fabricate entries.

1. [Datasets Used title / URL / author / year]
 2. [Datasets Used title / URL / author / year]
 3. [Datasets Used title / URL / author / year]
 4. [Datasets Used title / URL / author / year]
 5. [Datasets Used title / URL / author / year]
 6. [Datasets Used title / URL / author / year]
 7. [Datasets Used title / URL / author / year]
 8. [Datasets Used title / URL / author / year]
-

CITATIONS & DATA SOURCES**Citations - APIs & Web Services**

WHAT TO DO ON THIS PAGE

List every source under the category "APIs & Web Services" that you used for data in your project.

Format: Title of source, Author/Organization, Year, and full URL/link.

Only include sources you have actually referred to - do not fabricate entries.

1. [APIs & Web Services title / URL / author / year]
 2. [APIs & Web Services title / URL / author / year]
 3. [APIs & Web Services title / URL / author / year]
 4. [APIs & Web Services title / URL / author / year]
 5. [APIs & Web Services title / URL / author / year]
 6. [APIs & Web Services title / URL / author / year]
 7. [APIs & Web Services title / URL / author / year]
 8. [APIs & Web Services title / URL / author / year]
-

CITATIONS & DATA SOURCES**Citations - Research Papers & Journals**

WHAT TO DO ON THIS PAGE

List every source under the category "Research Papers & Journals" that you used for data in your project.

Format: Title of source, Author/Organization, Year, and full URL/link.

Only include sources you have actually referred to - do not fabricate entries.

1. [Research Papers & Journals title / URL / author / year]
 2. [Research Papers & Journals title / URL / author / year]
 3. [Research Papers & Journals title / URL / author / year]
 4. [Research Papers & Journals title / URL / author / year]
 5. [Research Papers & Journals title / URL / author / year]
 6. [Research Papers & Journals title / URL / author / year]
 7. [Research Papers & Journals title / URL / author / year]
 8. [Research Papers & Journals title / URL / author / year]
-

CITATIONS & DATA SOURCES**Citations - News Articles & Reports**

WHAT TO DO ON THIS PAGE

List every source under the category "News Articles & Reports" that you used for data in your project.

Format: Title of source, Author/Organization, Year, and full URL/link.

Only include sources you have actually referred to - do not fabricate entries.

1. [News Articles & Reports title / URL / author / year]
 2. [News Articles & Reports title / URL / author / year]
 3. [News Articles & Reports title / URL / author / year]
 4. [News Articles & Reports title / URL / author / year]
 5. [News Articles & Reports title / URL / author / year]
 6. [News Articles & Reports title / URL / author / year]
 7. [News Articles & Reports title / URL / author / year]
 8. [News Articles & Reports title / URL / author / year]
-

CITATIONS & DATA SOURCES**Citations - Government / Open Data Portals**

WHAT TO DO ON THIS PAGE

List every source under the category "Government / Open Data Portals" that you used for data in your project.

Format: Title of source, Author/Organization, Year, and full URL/link.

Only include sources you have actually referred to - do not fabricate entries.

1. [Government / Open Data Portals title / URL / author / year]
 2. [Government / Open Data Portals title / URL / author / year]
 3. [Government / Open Data Portals title / URL / author / year]
 4. [Government / Open Data Portals title / URL / author / year]
 5. [Government / Open Data Portals title / URL / author / year]
 6. [Government / Open Data Portals title / URL / author / year]
 7. [Government / Open Data Portals title / URL / author / year]
 8. [Government / Open Data Portals title / URL / author / year]
-

CITATIONS & DATA SOURCES**Citations - Images, Icons & Media Assets**

WHAT TO DO ON THIS PAGE

List every source under the category "Images, Icons & Media Assets" that you used for data in your project.

Format: Title of source, Author/Organization, Year, and full URL/link.

Only include sources you have actually referred to - do not fabricate entries.

1. [Images, Icons & Media Assets title / URL / author / year]
 2. [Images, Icons & Media Assets title / URL / author / year]
 3. [Images, Icons & Media Assets title / URL / author / year]
 4. [Images, Icons & Media Assets title / URL / author / year]
 5. [Images, Icons & Media Assets title / URL / author / year]
 6. [Images, Icons & Media Assets title / URL / author / year]
 7. [Images, Icons & Media Assets title / URL / author / year]
 8. [Images, Icons & Media Assets title / URL / author / year]
-

CITATIONS & DATA SOURCES**Citations - Surveys & Market Studies**

WHAT TO DO ON THIS PAGE

List every source under the category "Surveys & Market Studies" that you used for data in your project.

Format: Title of source, Author/Organization, Year, and full URL/link.

Only include sources you have actually referred to - do not fabricate entries.

1. [Surveys & Market Studies title / URL / author / year]
 2. [Surveys & Market Studies title / URL / author / year]
 3. [Surveys & Market Studies title / URL / author / year]
 4. [Surveys & Market Studies title / URL / author / year]
 5. [Surveys & Market Studies title / URL / author / year]
 6. [Surveys & Market Studies title / URL / author / year]
 7. [Surveys & Market Studies title / URL / author / year]
 8. [Surveys & Market Studies title / URL / author / year]
-

CITATIONS & DATA SOURCES**Citations - Other Data Sources**

WHAT TO DO ON THIS PAGE

List every source under the category "Other Data Sources" that you used for data in your project.

Format: Title of source, Author/Organization, Year, and full URL/link.

Only include sources you have actually referred to - do not fabricate entries.

1. [Other Data Sources title / URL / author / year]

2. [Other Data Sources title / URL / author / year]

3. [Other Data Sources title / URL / author / year]

4. [Other Data Sources title / URL / author / year]

5. [Other Data Sources title / URL / author / year]

6. [Other Data Sources title / URL / author / year]

7. [Other Data Sources title / URL / author / year]

8. [Other Data Sources title / URL / author / year]

REFERENCES & LEARNING RESOURCES

References - Frontend Framework / Library

WHAT TO DO ON THIS PAGE

List the resources (official docs, tutorials, courses, videos) used to learn "Frontend Framework / Library".

Format: Resource Title, Platform/Author, Year, and full URL/link.

This shows evaluators where your technical knowledge for this component came from.

1. [Frontend Framework / Library title / URL / author / year]
 2. [Frontend Framework / Library title / URL / author / year]
 3. [Frontend Framework / Library title / URL / author / year]
 4. [Frontend Framework / Library title / URL / author / year]
 5. [Frontend Framework / Library title / URL / author / year]
 6. [Frontend Framework / Library title / URL / author / year]
 7. [Frontend Framework / Library title / URL / author / year]
 8. [Frontend Framework / Library title / URL / author / year]
-

REFERENCES & LEARNING RESOURCES**References - Backend Framework**

WHAT TO DO ON THIS PAGE

List the resources (official docs, tutorials, courses, videos) used to learn "Backend Framework".

Format: Resource Title, Platform/Author, Year, and full URL/link.

This shows evaluators where your technical knowledge for this component came from.

1. [Backend Framework title / URL / author / year]
 2. [Backend Framework title / URL / author / year]
 3. [Backend Framework title / URL / author / year]
 4. [Backend Framework title / URL / author / year]
 5. [Backend Framework title / URL / author / year]
 6. [Backend Framework title / URL / author / year]
 7. [Backend Framework title / URL / author / year]
 8. [Backend Framework title / URL / author / year]
-

REFERENCES & LEARNING RESOURCES**References - Programming Languages**

WHAT TO DO ON THIS PAGE

List the resources (official docs, tutorials, courses, videos) used to learn "Programming Languages".

Format: Resource Title, Platform/Author, Year, and full URL/link.

This shows evaluators where your technical knowledge for this component came from.

1. [Programming Languages title / URL / author / year]
 2. [Programming Languages title / URL / author / year]
 3. [Programming Languages title / URL / author / year]
 4. [Programming Languages title / URL / author / year]
 5. [Programming Languages title / URL / author / year]
 6. [Programming Languages title / URL / author / year]
 7. [Programming Languages title / URL / author / year]
 8. [Programming Languages title / URL / author / year]
-

REFERENCES & LEARNING RESOURCES

References - Database Technology

WHAT TO DO ON THIS PAGE

List the resources (official docs, tutorials, courses, videos) used to learn "Database Technology".

Format: Resource Title, Platform/Author, Year, and full URL/link.

This shows evaluators where your technical knowledge for this component came from.

1. [Database Technology title / URL / author / year]
 2. [Database Technology title / URL / author / year]
 3. [Database Technology title / URL / author / year]
 4. [Database Technology title / URL / author / year]
 5. [Database Technology title / URL / author / year]
 6. [Database Technology title / URL / author / year]
 7. [Database Technology title / URL / author / year]
 8. [Database Technology title / URL / author / year]
-

REFERENCES & LEARNING RESOURCES**References - Cloud / Hosting Platform**

WHAT TO DO ON THIS PAGE

List the resources (official docs, tutorials, courses, videos) used to learn "Cloud / Hosting Platform".

Format: Resource Title, Platform/Author, Year, and full URL/link.

This shows evaluators where your technical knowledge for this component came from.

1. *[Cloud / Hosting Platform title / URL / author / year]*
 2. *[Cloud / Hosting Platform title / URL / author / year]*
 3. *[Cloud / Hosting Platform title / URL / author / year]*
 4. *[Cloud / Hosting Platform title / URL / author / year]*
 5. *[Cloud / Hosting Platform title / URL / author / year]*
 6. *[Cloud / Hosting Platform title / URL / author / year]*
 7. *[Cloud / Hosting Platform title / URL / author / year]*
 8. *[Cloud / Hosting Platform title / URL / author / year]*
-

REFERENCES & LEARNING RESOURCES**References - AI / ML Libraries & Frameworks**

WHAT TO DO ON THIS PAGE

List the resources (official docs, tutorials, courses, videos) used to learn "AI / ML Libraries & Frameworks".

Format: Resource Title, Platform/Author, Year, and full URL/link.

This shows evaluators where your technical knowledge for this component came from.

1. [AI / ML Libraries & Frameworks title / URL / author / year]
 2. [AI / ML Libraries & Frameworks title / URL / author / year]
 3. [AI / ML Libraries & Frameworks title / URL / author / year]
 4. [AI / ML Libraries & Frameworks title / URL / author / year]
 5. [AI / ML Libraries & Frameworks title / URL / author / year]
 6. [AI / ML Libraries & Frameworks title / URL / author / year]
 7. [AI / ML Libraries & Frameworks title / URL / author / year]
 8. [AI / ML Libraries & Frameworks title / URL / author / year]
-

REFERENCES & LEARNING RESOURCES**References - Computer Vision / NLP Tools**

WHAT TO DO ON THIS PAGE

List the resources (official docs, tutorials, courses, videos) used to learn "Computer Vision / NLP Tools".

Format: Resource Title, Platform/Author, Year, and full URL/link.

This shows evaluators where your technical knowledge for this component came from.

1. [Computer Vision / NLP Tools title / URL / author / year]
 2. [Computer Vision / NLP Tools title / URL / author / year]
 3. [Computer Vision / NLP Tools title / URL / author / year]
 4. [Computer Vision / NLP Tools title / URL / author / year]
 5. [Computer Vision / NLP Tools title / URL / author / year]
 6. [Computer Vision / NLP Tools title / URL / author / year]
 7. [Computer Vision / NLP Tools title / URL / author / year]
 8. [Computer Vision / NLP Tools title / URL / author / year]
-

REFERENCES & LEARNING RESOURCES**References - Version Control & DevOps Tools**

WHAT TO DO ON THIS PAGE

List the resources (official docs, tutorials, courses, videos) used to learn "Version Control & DevOps Tools".

Format: Resource Title, Platform/Author, Year, and full URL/link.

This shows evaluators where your technical knowledge for this component came from.

1. [Version Control & DevOps Tools title / URL / author / year]
 2. [Version Control & DevOps Tools title / URL / author / year]
 3. [Version Control & DevOps Tools title / URL / author / year]
 4. [Version Control & DevOps Tools title / URL / author / year]
 5. [Version Control & DevOps Tools title / URL / author / year]
 6. [Version Control & DevOps Tools title / URL / author / year]
 7. [Version Control & DevOps Tools title / URL / author / year]
 8. [Version Control & DevOps Tools title / URL / author / year]
-

REFERENCES & LEARNING RESOURCES**References - Testing Tools & Frameworks**

WHAT TO DO ON THIS PAGE

List the resources (official docs, tutorials, courses, videos) used to learn "Testing Tools & Frameworks".

Format: Resource Title, Platform/Author, Year, and full URL/link.

This shows evaluators where your technical knowledge for this component came from.

1. [Testing Tools & Frameworks title / URL / author / year]
 2. [Testing Tools & Frameworks title / URL / author / year]
 3. [Testing Tools & Frameworks title / URL / author / year]
 4. [Testing Tools & Frameworks title / URL / author / year]
 5. [Testing Tools & Frameworks title / URL / author / year]
 6. [Testing Tools & Frameworks title / URL / author / year]
 7. [Testing Tools & Frameworks title / URL / author / year]
 8. [Testing Tools & Frameworks title / URL / author / year]
-

REFERENCES & LEARNING RESOURCES**References - UI/UX Design Tools**

WHAT TO DO ON THIS PAGE

List the resources (official docs, tutorials, courses, videos) used to learn "UI/UX Design Tools".

Format: Resource Title, Platform/Author, Year, and full URL/link.

This shows evaluators where your technical knowledge for this component came from.

1. [UI/UX Design Tools title / URL / author / year]
 2. [UI/UX Design Tools title / URL / author / year]
 3. [UI/UX Design Tools title / URL / author / year]
 4. [UI/UX Design Tools title / URL / author / year]
 5. [UI/UX Design Tools title / URL / author / year]
 6. [UI/UX Design Tools title / URL / author / year]
 7. [UI/UX Design Tools title / URL / author / year]
 8. [UI/UX Design Tools title / URL / author / year]
-

REFERENCES & LEARNING RESOURCES**References - Security & Authentication**

WHAT TO DO ON THIS PAGE

List the resources (official docs, tutorials, courses, videos) used to learn "Security & Authentication".

Format: Resource Title, Platform/Author, Year, and full URL/link.

This shows evaluators where your technical knowledge for this component came from.

1. *[Security & Authentication title / URL / author / year]*
 2. *[Security & Authentication title / URL / author / year]*
 3. *[Security & Authentication title / URL / author / year]*
 4. *[Security & Authentication title / URL / author / year]*
 5. *[Security & Authentication title / URL / author / year]*
 6. *[Security & Authentication title / URL / author / year]*
 7. *[Security & Authentication title / URL / author / year]*
 8. *[Security & Authentication title / URL / author / year]*
-

REFERENCES & LEARNING RESOURCES**References - Tutorials, Courses & Documentation**

WHAT TO DO ON THIS PAGE

List the resources (official docs, tutorials, courses, videos) used to learn "Tutorials, Courses & Documentation".

Format: Resource Title, Platform/Author, Year, and full URL/link.

This shows evaluators where your technical knowledge for this component came from.

1. [Tutorials, Courses & Documentation title / URL / author / year]
 2. [Tutorials, Courses & Documentation title / URL / author / year]
 3. [Tutorials, Courses & Documentation title / URL / author / year]
 4. [Tutorials, Courses & Documentation title / URL / author / year]
 5. [Tutorials, Courses & Documentation title / URL / author / year]
 6. [Tutorials, Courses & Documentation title / URL / author / year]
 7. [Tutorials, Courses & Documentation title / URL / author / year]
 8. [Tutorials, Courses & Documentation title / URL / author / year]
-